

CONSTRUCTORES - GETTER Y SETTER SIEMPRE PÚBLICOS SI VAMOS A HEREDAR

PROBAR SIEMPRE EN LOS HIJOS

1. Iniciar un fichero Java:

```
public class nombre{  
    Contenido del fichero  
}
```

2. Declaración de atributos:

Cuando declaramos variables en Java habrá que indicar siempre su tipo. Tenemos los siguientes:

- int = Números enteros
- Str = String
- integer = Números enteros (incluidos negativos)
- double = Numero con coma
- valor [] = Array -> int[] lista

Tenemos varias formas de declarar variables:

- public = Públicas
- private = Privada

EJEMPLOS:

- private int codigoCentro;
- private String nomeCentro;
- public double y;
- String[] coches = {"Toyota", "Ford", "Mazda", "BMW"};

2.1. Declaración de atributos:

EJEMPLOS COMPLETOS:

```
public class Hora {  
    public int horas;  
    public int minutos;
```

```
public int segundos;
```

El que sea público indica que se puede acceder a esta clase desde el lugar del proyecto.

A esas “variables” se les llama atributos. Almacenan la información que le indiquemos.

3. Constructor:

Su función es inicializar los atributos que tengamos para que estos tengan una función posteriormente.

EJEMPLO:

```
public Hora(int horas, int minutos, int segundos) {  
    this.horas = horas;  
    this.minutos = minutos;  
    this.segundos = segundos;  
}
```

public Hora = Este es el constructor. Debe tener el mismo nombre que el fichero. Este no tiene retorno, como se mencionó su única función es inicializar los atributos que están entre paréntesis.

this.horas = horas; = Esta línea significa que estamos recogiendo el valor del parámetro que está en el paréntesis (que es solo temporal y se encuentra a la derecha del signo igual) y lo estamos guardando en el atributo de la clase (que está a la izquierda del signo igual). Al poner "this.horas" a la izquierda, aclaramos que queremos guardarlo ahí de forma permanente para utilizarlo posteriormente en las operaciones..

ACLARACIÓN ANTES DE LOS SIGUIENTES PUNTOS - DIFERENCIAS ENTRE INICIALIZADOR Y SÓLO “VOID”:

- void = Método sin retorno, es decir, nunca va a devolver nada, simplemente sirve para hacer ciertas modificaciones que en ningún momento se van a mostrar.

4. Inicializador:

Sirve para poner en funcionamiento todo el diseño de la clase. Este crea un objeto con el que se pondrá a prueba el funcionamiento del diseño del programa.

EJEMPLO:

```
public static void main(String [] args) {  
    Hora h1 = new Hora(12.23);  
    System.out.println(h1.getHoras());  
    System.out.println(h1.getMinutos());  
    System.out.println(h1.getSegundos());  
}
```

5. Getter:

Sirve para consultar o leer de forma segura el valor de un atributo privado.

EJEMPLO:

```
public String getCurso() { // cualquier otra clase puede llamar a este método para  
pedir el dato  
    return this.curso; // Al poner el return indicamos que cuando llamemos a este  
método nos va a devolver el valor de this.curso
```

6. Setter:

Sirve para modificar, cambiar o asignar un nuevo valor al atributo this.curso. Cualquier otra clase podrá llamar a este método para cambiar this.curso. Entre paréntesis siempre tendremos el nombre del atributo.

EJEMPLO:

```
public void setCurso(String curso) { // Se utiliza la palabra clave void porque su única  
función es guardar un dato, por lo que no necesita devolver ninguna respuesta  
    this.curso = curso;  
}
```

7. Métodos:

7.1. Metodo de comportamiento:

Modifican las variables en función a la instrucción que le demos. También sirven para hacer cálculos y otras funciones que no modifiquen directamente la variable, pero si realizan algún tipo de acción.

EJEMPLO:

```
public void avanzarUnMinuto() {  
    this.minutos = this.minutos + 1; // Modifica los atributos permanentes  
}
```

7.2. Métodos estáticos:

No necesitan nada fuera de ellos, solo con su creación ya podemos realizar operaciones

EJEMPLO:

```
public static int sumar(int a, int b) {  
    return a + b; // Funciona de forma aislada sin depender de atributos. En el ejemplo  
    las variables en el paréntesis no están creadas anteriormente, si no directamente en  
    el método.  
}
```

ACLARACIÓN: @OVERRIDE NO ES UN MÉTODO, SI NO UNA ANOTACIÓN QUE CUENTA CON MÉTODOS, PERO POR LAS SIMILITUDES LO AÑADO AQUÍ, Y PORQUE COMO DIJE SI QUE LOS CONTIENE.

7.2. @overreide:

Cuenta con 2 métodos:

- toString()= Sirve para transformar el objeto en un texto legible. Si no lo creas, al imprimir un objeto verás un código raro en la pantalla.

EJEMPLO:

@Override

```
public String toString() {  
    return this.horas + ":" + this.minutos + ":" + this.segundos;  
}
```

- equals(object obj)= Sirve para comparar si dos objetos diferentes contienen exactamente los mismos datos.

EJEMPLO:

```
public class Hora {  
    public int horas;  
    public int minutos;
```

```

public int segundos;

// Constructor (el que ya conoces)
public Hora(int horas, int minutos, int segundos) {
    this.horas = horas;
    this.minutos = minutos;
    this.segundos = segundos;
}

@Override
public boolean equals(Object obj) {
    // 1. Si apuntan a la misma dirección de memoria, son el mismo objeto
    if (this == obj) return true;

    // 2. Si el otro objeto es nulo o no es de la clase Hora, no son iguales
    if (obj == null || getClass() != obj.getClass()) return false;

    // 3. Convertimos el objeto genérico "obj" en un objeto tipo "Hora"
    Hora otraHora = (Hora) obj;

    // 4. Comparamos si todos sus atributos permanentes son idénticos
    return this.horas == otraHora.horas &&
        this.minutos == otraHora.minutos &&
        this.segundos == otraHora.segundos;
}
}

```

8. interface=

Sirve como una plantilla o para obligar a que cualquier clase geométrica que implemente esta interfaz (como un Cuadrado o un Circulo) tenga la capacidad de calcular su área. Al usar default, se proporciona una respuesta estándar (return 0;) para aquellas clases que no escriban su propia fórmula matemática.

EJEMPLO:

```

public interface Area {

```

```

default double calcularArea(){
    return 0;
}
}

```

MÉTODO ADICIONAL POR SI ACASO (CASI SEGURO NO ENTRA)

3. Métodos Recursivos

Son métodos que, para resolver un problema, se llaman a sí mismos una y otra vez dentro de su propio código. Funcionan de forma parecida a un bucle.

- **¿Para qué sirven?:** Se utilizan para resolver problemas matemáticos complejos o búsquedas (como el cálculo del factorial de un número o la sucesión de Fibonacci) dividiendo el problema en partes más pequeñas.
- **Regla de oro:** Siempre deben tener un "caso base" (una condición de salida) para no quedarse repitiéndose infinitamente y romper la memoria del ordenador.

java

```

public int calcularFactorial(int n) {
    if (n == 1) {
        return 1; // Caso base: aquí se detiene la repetición
    }
    return n * calcularFactorial(n - 1); // El método se llama a sí mismo
}

```

9. Operadores lógicos:

- &&= and
- || = or
- != not

COMANDOS

- extends= Significa que estás creando una clase nueva que adquiere automáticamente todas las características (atributos) y comportamientos (métodos) de una clase existente (clase padre)

EJEMPLO:

```

public class Alumno extends MembroComunidadeEscolar {

```

```

private String curso;
private int asignaturas;
public Alumno (String nome, String dni, int edad, double estatura, int codCentro,
String nomCentro, String curso, int asignaturas){
    super(nome, dni, edad, estatura, codCentro, nomCentro);
    this.curso = curso;
    this.asignaturas = asignaturas;
} //Constructor con las clases propias e importadas. En el super pondremos las
importadas. Las que tienen this serán las propias.

```

```

public Alumno() {
    super();
    this.curso = "desconocido";
    this.asignaturas = 0; // Valores que le añadimos
}

```

- if y sus iguales=

EJEMPLO (no necesita mucha explicación):

int temperatura = 18; // Puedes cambiar este número para probar los diferentes caminos

```

if (temperatura >= 30) {
    // Se ejecuta solo si la condición es VERDADERA
    System.out.println("Hace mucho calor.");
}
else if (temperatura >= 15) {
    // Se ejecuta si la primera fue falsa, pero esta es VERDADERA
    System.out.println("El clima está agradable.");
}
else {
    // Se ejecuta automáticamente si TODAS las condiciones anteriores fueron
    FALSAS
    System.out.println("Hace frío.");
}

```

- Escritura= Entrecomillada, y si vamos a añadir algún atributo cerramos las comillas y ponemos + "atributo".

EJEMPLO:

```
return "Alumno=" + Curso: " + curso + ""
```

- Creación de objeto en el inicializador= NombreDeLaClase "variable" = new NombreDeLaClase("atributos de la clase"). Si son String entre comillas, números sin ellas

EJEMPLO:

```
Alumno a2 = new Alumno("Pepe", "12345678A", 20, 1.65, 27001, "IES San Clemente", "DAM2", 7);
```

- print en terminal= System.out.println("lo que queramos escribir" + variableDeclaradaPuntoAnterior.geter)

EJEMPLO TOMANDO DE BASE EL PUNTO ANTERIOR:

```
System.out.println(a2.getNome() + " " + a2.getDni() + " " + a2.getEdade() + " ");
System.out.println(a2); //printear todo
```

- Bucle while= Sirve para repetir un bloque de código de forma automática tantas veces como sea necesario, siempre y cuando se siga cumpliendo la condición que tiene entre paréntesis.

EJEMPLO:

```
int contador = 0;
while (contador <= 5) {
    System.out.println(contador);
    contador++; //Va a ir aumentando el valor de uno en uno por cada iteración del bucle.
}
```

- import java.util.Scanner= Sirve para traer y habilitar en tu archivo de código una herramienta externa de Java llamada Scanner. sirve para que el usuario pueda escribir datos a través del teclado y el programa los lea. Hay que importarla al principio del archivo.

EJEMPLO - PANTALLA DE INICIO CON SELECCIÓN DE OPCIONES:

```
Scanner teclado = new Scanner(System.in);
int opcion = 0;
while (opcion != 3) {
    System.out.println("Elije una opcion");
    System.out.println("1 Saludar");
    System.out.println("2 Despedir");
    System.out.println("3 Sair");
    String op = teclado.nextLine();
```

- Bucle for= Igual que el while pero con diferente sintaxis.

EJEMPLO:

```
for ( int i = 5; i < 20; i +=3) {
    System.out.println("Indice: "+i); }
}
```

- switch()= Complementario al apartado anterior sirve para establecer una acción dependiendo de la opción que elijamos.

EJEMPLO:

```
switch (opcion) {
    case 1:
        opcion++;
        System.out.println("Saludos");
        break; // Para que no pase automaticamente a la siguiente opción
    case 2:
        opcion++;
        System.out.println("Despedir");
    case 3:
        opcion++;
        System.out.println("Saliendo");
        break;
```

- implements= Obliga a una clase a escribir el código de los métodos definidos que son: calcularArea(), calcularVolumen() y calcularPerimetro().

EJEMPLO:

```
public class Cilindro2 extends Circulo implements Area, Volumen, Perimetro {  
    private double altura;
```

```
  
    public Cilindro2(int radio, double altura) {  
        super(radio);  
        this.altura = altura;  
    }  

```

@Override

```
    public double calcularArea() { // Este es el método que vamos a implementar. Esta  
    anteriormente explicado en el punto implements  
        return 2 * Math.PI * getRadio() * (getRadio() + altura);  
    }  

```

**ACLARACIÓN PARA LOS SIGUIENTES PUNTOS: PARA ACCEDER A LOS
ELEMENTOS DE UN ARRAY TENDREMOS QUE EFECTUAR LAS SIGUIENTES
ACCIONES:**

1. lista[0]...
2. En bucle = lista[i] (nombre que se le suele asociar a dicha variable en un bucle for).

- char Crea una lista con los caracteres que establezcamos. Podremos ir rellenandolos.

EJEMPLO:

```
public static void main (String [] args) {  
    char[] letras = new char[10];  
    letras[0] = 'a';  
    letras[1] = 'b';  
    nomes[2] = new String("Ana"); // Misma función que los anteriores
```

- char no es el único con el que se puede hacer eso, también podemos hacerlo de la siguiente forma: String[] nomes = new String[10]; (por supuesto con otro tipo de datos).

- `.length`= Numerar los valores de una lista.
- `import java.util.ArrayList`= un `ArrayList` puede cambiar de tamaño automáticamente. Puedes añadir (`.add()`) o eliminar (`.remove()`) elementos en cualquier momento del programa sin preocuparte por el límite de espacio inicial.

EJEMPLO:

// 1. Declarar un `ArrayList` de textos (`String`)

// Nace completamente vacío, con un tamaño inicial de 0

```
ArrayList<String> alumnos = new ArrayList<>();
```

// 2. AÑADIR ELEMENTOS con `.add()`

// El `ArrayList` expande su tamaño automáticamente con cada inserción

```
alumnos.add("Carlos");
```

```
alumnos.add("Ana");
```

```
alumnos.add("Luis");
```

```
System.out.println("Lista inicial (Tamaño " + alumnos.size() + "): " + alumnos);
```

// Imprime: Lista inicial (Tamaño 3): [Carlos, Ana, Luis]

// 3. ELIMINAR ELEMENTOS con `.remove()`

// Opción A: Eliminar por el nombre exacto (objeto)

```
alumnos.remove("Ana");
```

// Opción B: Eliminar por su posición / índice (el primer elemento es el 0)

```
alumnos.remove(0); // Elimina a "Carlos", que estaba en la posición 0
```

// 4. Resultado final

// El `ArrayList` se encoge automáticamente y reorganiza sus índices

```
System.out.println("Lista final (Tamaño " + alumnos.size() + "): " + alumnos);
```

// Imprime: Lista final (Tamaño 1): [Luis]

- `java.util.Iterator`= Recorre colecciones de datos. Funciona como un "puntero" que va pasando elemento por elemento dentro de una lista. Su mayor ventaja

frente a un bucle for tradicional es que permite eliminar elementos de la lista de forma segura mientras la estás recorriendo.

EJEMPLO:

```
import java.util.ArrayList;
```

```
import java.util.Iterator; // Importación obligatoria
```

```
public class Ejemplolterator {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<Integer> numeros = new ArrayList<>();
```

```
        numeros.add(10);
```

```
        numeros.add(15); // Impar (queremos eliminarlo)
```

```
        numeros.add(20);
```

```
        numeros.add(25); // Impar (queremos eliminarlo)
```

```
        numeros.add(30);
```

```
        System.out.println("Lista original: " + numeros);
```

```
        // 1. Creamos el Iterator solicitándolo a la lista
```

```
        Iterator<Integer> puntero = numeros.iterator();
```

```
        // 2. Iniciamos el bucle "Mientras el puntero detecte que HAY UN SIGUIENTE elemento"
```

```
        while (puntero.hasNext()) {
```

```
            // 3. Movemos el puntero al siguiente elemento y recogemos su valor
```

```
            int numeroActual = puntero.next();
```

```
            // 4. Si el número es impar, lo eliminamos de forma segura
```

```
            if (numeroActual % 2 != 0) {
```

```
                puntero.remove(); // Borra de la lista el último elemento que devolvió
```

```
            .next()
```

```
        }
```

```
    }
```

```

        System.out.println("Lista filtrada (solo pares): " + numeros);
        // Imprime: Lista filtrada (solo pares): [10, 20, 30]
    }
}

```

- `.split()`= Coge los elementos de un String, y los separa con el carácter que le establezcamos, pasandolos así a un formato de lista.

EJEMPLO:

Java

```

String hora = "08:45";
String[] partes = hora.split(":");

```

- **Antes del `.split()`** : Tienes un solo objeto de tipo texto (`"08:45"`).
- **Después del `.split()`** : Tienes un arreglo con dos elementos independientes (`"08"` y `"45"`).

- `parseInt()`= Pasa un número en un String a número real (carácter numérico).

EJEMPLO:

```
this.horas = Integer.parseInt(partes[0]);
```

- Cambiar de tipo de dato (en especial numéricos)= `this.horas = (int) hora;`
- `Arrays.sort()`= Ordena en sentido ascendente una lista.
- `Arrays.toString()`= Pasa una lista String, separando los valores con comas.
- `import java.util.Random;`= Permite importar números random

EJEMPLO:

```

Random generador = new Random(); // 2. Creamos el objeto generador
// 3. Le pedimos un número entero aleatorio entre 0 y 9

```

```
int numeroAlAzar = generador.nextInt(10);
```

- `.compare=` Sirve para comparar dos números decimales. Funciona devolviendo un número entero basado en tres reglas:
 1. Si el primero es menor, devuelve un número negativo (-1).
 2. Si el primero es mayor, devuelve un número positivo (1).
 3. Si ambos son exactamente iguales, devuelve un 0.

EJEMPLO:

```
return Double.compare(this.getEstatura(), outroTraballador.getEstatura()) == 0;  
// == 0: Es la condición. Al final de la línea se pregunta: "¿El resultado de la  
comparación fue cero?". Si la respuesta es sí, todo el bloque se convierte en true.
```

- `\n=` Hacer un salto de línea en un texto.

CONCEPTO CLAVE= EL ENCAPSULAMIENTO CONSISTE EN 3 COSAS:

1. HACER MÉTODOS Y ATRIBUTOS PRIVADOS
2. GETTER
3. SETTER

CONCEPTOS EXTRA:

1. El bloque `try-catch` (Control de Excepciones)

Como usas `Integer.parseInt()`, el profesor te va a pedir casi seguro que protejas el programa por si el usuario introduce una letra en vez de un número.

Java

```
try {  
    this.horas = Integer.parseInt(partes[0]);  
} catch (NumberFormatException e) {  
    System.out.println("El texto no era un número válido. Se asigna 0.");  
    this.horas = 0;  
}
```

En tus apuntes explicaste genial el uso de `super()` dentro del constructor de `Alumno`. Pero recuerda que `super` también sirve para **llamar a un método del padre que has sobrescrito**.

Por ejemplo, si quieres usar el `toString()` del padre y añadirle algo:

Java

```
@Override
public String toString() {
    return super.toString() + " - Curso: " + this.curso;
    // super.toString() ejecuta el código de MembroComunidadeEscolar
}
```

POLIMORFISMO

- Permite tratar objetos de diferentes clases como si fuera una sola clase invocando un solo método y obteniendo valores distintos.

EJEMPLO:

CLASE PADRE:

```
public class Vehiculo {
    public void arrancar() {
        System.out.println("El vehículo está arrancando...");
    }
}
```

CLASES HIJAS:

```
public class Coche extends Vehiculo {
    @Override
    public void arrancar() {
        System.out.println("Girar llave: ¡Brum, brum! El motor de gasolina ruge.");
    }
}
```

```
public class MotoElectrica extends Vehiculo {
    @Override
    public void arrancar() {
```

```
        System.out.println("Pulsar botón: Sistema eléctrico encendido en silencio.");
    }
}
```

EJECUCIÓN:

```
public static void main(String[] args) {
    // Creamos una lista genérica de Vehículos (pueden entrar coches, motos, etc.)
    ArrayList<Vehiculo> garaje = new ArrayList<>();

    garaje.add(new Coche());
    garaje.add(new MotoElectrica());

    // Recorremos la lista con un for-each
    for (Vehiculo v : garaje) {
        v.arrancar(); // 🌟 AQUÍ ACTÚA EL POLIMORFISMO
    }
}
```

RESULTADO:

Girar llave: ¡Brum, brum! El motor de gasolina ruge.

Pulsar botón: Sistema eléctrico encendido en silencio.

INSTANCIAR=Hacer new

ABSTRACCIÓN

- Es una clase que no se puede instanciar directamente porque está incompleta. Por así decirlo es solo una plantilla que otras clases cogen y completan.

EJEMPLO:

CLASE PADRE:

```
public abstract class Vehiculo {  
    // Todos los vehículos tienen una marca (característica esencial)  
    private String marca;  
  
    // Método abstracto: Obliga a las clases hijas a definir CÓMO frenan,  
    // pero aquí en el padre no nos importan los detalles.  
    public abstract void frenar();  
}
```

CLASE HIJA:

```
public class CocheDeCarreras extends Vehiculo {  
    @Override  
    public void frenar() {  
        // Aquí se escriben los detalles complejos que al resto del programa no le  
        importan  
        System.out.println("Activando frenos de disco cerámicos de alta potencia.");  
    }  
}
```

MODULARIDAD= Hacer el código en varios ficheros

DESTRUCTOR

- Un destructor es un método especial que se ejecuta automáticamente justo antes de que un objeto sea destruido y borrado por completo. Absolutamente todas las clases heredan `Object`, y esta trae el método `finalize()` que viene vacío. Si se quiere que haga algo habrá que utilizar `@Override`.

EJEMPLO:

```
public class ArchivoTexto {
    private String nombreArchivo;

    public ArchivoTexto(String nombre) {
        this.nombreArchivo = nombre;
        System.out.println("Archivo " + nombreArchivo + " abierto.");
    }

    // El "Destructor" en Java (Sobrescribiendo finalize)
    @Override
    protected void finalize() throws Throwable {
        try {
            // 1. Limpiar recursos que no son de Java (Cerrar el archivo)
            System.out.println("Limpiando recursos: Cerrando el archivo " +
nombreArchivo);
        } finally {
            // 2. Obligatorio llamar al finalize del padre (Object)
            super.finalize();
        }
    }
}
```

CONVERTIR A CONSTANTE= `public static final int MODEL_NUM = 883;`

JAVA COLLECTION FRAMEWORK

- Objetos que permiten agrupar múltiples elementos dentro de una sola unidad, con la enorme ventaja de que son dinámicas: crecen y se encogen solas en memoria según añades o quitas cosas. Sus elementos principales son:
 1. ArrayList
 2. Set: No permite elementos duplicados. Si intentas añadir un elemento que ya existe, Java simplemente lo ignora. Además, por defecto, no garantizan el orden de inserción.

EJEMPLO:

```
import java.util.HashSet;
```

```
public class EjemploSet {  
    public static void main(String[] args) {  
        // Creamos el conjunto de DNIs (tienen que ser únicos)  
        HashSet<String> conjuntoDnis = new HashSet<>();  
  
        conjuntoDnis.add("12345678A");  
        conjuntoDnis.add("87654321B");  
        conjuntoDnis.add("12345678A"); // 👉 Duplicado intentando entrar...  
  
        // ❌ Aquí NO existe el .get(0). No puedes pedir una posición.  
        // Para saber si un elemento está, usas .contains()  
        if (conjuntoDnis.contains("12345678A")) {  
            System.out.println("El DNI 12345678A ya está registrado.");  
        }  
  
        // Imprimir todo el conjunto  
        System.out.println("Conjunto de DNIs: " + conjuntoDnis);  
    }  
}
```

3. Map: Guarda la información en parejas de Clave - Valor.
mapa.put("Pepe", 666111222); y para recuperarlo usas
mapa.get("Pepe")

Casi todas las colecciones (especialmente `List` y `Set`) comparten los mismos métodos, lo que hace que cambiar de una a otra sea muy fácil:

- `.add()` : Añade un elemento a la colección.
- `.remove()` : Elimina un elemento.
- `.size()` : Te dice cuántos elementos hay dentro (el equivalente a `.length` en arrays).
- `.clear()` : Borra todo lo que hay dentro de la colección, dejándola vacía.
- `.isEmpty()` : Devuelve `true` si la colección no tiene ningún elemento.
- `.contains()` : Devuelve `true` si el elemento que buscas está dentro de la lista (es genial para buscar cosas rápido sin hacer un bucle `for`).

EJEMPLO:

```
import java.util.HashMap;
```

```
public class EjemploMap {
```

```
    public static void main(String[] args) {
```

```
        // Creamos un diccionario/agenda: Clave (DNI) -> Valor (Nombre de Alumno)
```

```
        HashMap<String, String> agendaAlumnos = new HashMap<>();
```

```
        // Añadimos datos con .put(clave, valor)
```

```
        agendaAlumnos.put("12345678A", "Ana García");
```

```
        agendaAlumnos.put("87654321B", "Carlos López");
```

```
        agendaAlumnos.put("55555555C", "Pepe Soneira");
```

```
        // Recuperamos un dato usando su clave con .get(clave)
```

```
        String alumnoEncontrado = agendaAlumnos.get("87654321B");
```

```
        System.out.println("El alumno con DNI 87654321B es: " +  
alumnoEncontrado);
```

```
        // Imprimir todo el mapa  
        System.out.println("Agenda completa: " + agendaAlumnos);  
    }  
}
```

EJEMPLOS EN ARRAY LISTA

```
import java.util.ArrayList;
```

```
public class PruebaMetodos {  
    public static void main(String[] args) {  
        ArrayList<String> lista = new ArrayList<>();  
  
        // 📦 Probando add()  
        System.out.println(lista.add("Moto")); // Imprime: true  
        lista.add("Coche");  
  
        // 🔍 Probando size() e isEmpty()  
        System.out.println("¿Está vacía?: " + lista.isEmpty()); // Imprime: false  
        System.out.println("Tamaño actual: " + lista.size()); // Imprime: 2  
  
        // 🔍 Probando contains()  
        if (lista.contains("Coche")) {  
            System.out.println("¡El coche está en la lista!");  
        }  
  
        // 🗑️ Probando remove()  
        boolean borrado = lista.remove("Moto");  
        System.out.println("¿Se borró la moto?: " + borrado); // Imprime: true
```

```
// 🗑️ Probando clear()
lista.clear();
System.out.println("Tamaño tras usar clear(): " + lista.size()); // Imprime:
0
}
}
```

MAS EJEMPLOS:

```
ArrayList<String> nomes;//Declarar variable ArrayList
nomes = new ArrayList();//Intanciar ArrayList
nomes.add("Xaime");//Engadir elementos
nomes.add("Gustavo");
nomes.add("Alicia");
nomes.add("Jose");
nomes.add(2,"Paco");
String str=nomes.get(0);
System.out.println(str);
nomes.remove(0); //Recuperar o valor índice 0
nomes.remove(nomes.size() - 1);//Elimina derradeiro
nomes.remove("Gustavo");//Elimina o elemento
System.out.println(nomes);//Visualiza a lista
```

LISTEITOR EJEMPLO

🖥️ Análisis del código línea a línea

1. Declaración e Instanciación

Java

```
ArrayList<String> nomes; // Declarar variable ArrayList
nomes = new ArrayList(); // Instanciar ArrayList
```

Qué hace: Crea la variable en memoria y luego reserva el espacio para el objeto (new).

Nota para el examen: Aunque Java lo permite, hoy en día se considera mejor práctica instanciarlo usando los operadores de diamante: `nomes = new ArrayList<>();`.

2. Añadir elementos básicos

Java

```
nomes.add("Xaime"); // Posición 0
```

```
nomes.add("Gustavo"); // Posición 1
```

```
nomes.add("Alicia"); // Posición 2
```

```
nomes.add("Jose"); // Posición 3
```

Estado de la lista en este momento: ["Xaime", "Gustavo", "Alicia", "Jose"]

3. Inserción en una posición intermedia (¡Ojo con esto!)

Java

```
nomes.add(2, "Paco");
```

Qué hace: Obliga a Java a meter a "Paco" exactamente en el índice 2. Los elementos que estaban de ahí en adelante ("Alicia" y "Jose") no se borran, sino que se desplazan una posición a la derecha.

Estado de la lista: ["Xaime", "Gustavo", "Paco", "Alicia", "Jose"]

4. Recuperar un valor

Java

```
String str = nomes.get(0); // Guarda "Xaime" en la variable str
```

```
System.out.println(str); // Pinta "Xaime" en la consola
```

5. Eliminación por Índice y Dinámica

Java

```
nomes.remove(0); // ✗ Elimina al elemento del índice 0 ("Xaime")
```

Nota sobre tu comentario: En tu apunte pusiste "Recuperar o valor índice 0". ¡Cuidado! `.remove(0)` lo elimina de la lista (aunque es cierto que el método te devuelve el nombre que acaba de borrar por si quieres guardarlo, su función principal es destruir la posición).

Al borrar el índice 0, todos los demás elementos se vuelven a desplazar hacia la izquierda para rellenar el hueco. El índice 0 ahora pasa a ser "Gustavo".

Estado de la lista: ["Gustavo", "Paco", "Alicia", "Jose"]

Java

`nomes.remove(nomes.size() - 1);` // ✗ Elimina el último elemento de la lista

Qué hace: Esta fórmula matemática (`size() - 1`) sirve para encontrar el último índice válido de cualquier lista de forma dinámica. Como el tamaño actual es 4, borra el índice 3, que corresponde a "Jose".

Estado de la lista: ["Gustavo", "Paco", "Alicia"]

6. Eliminación por Objeto / Texto exacto

Java

`nomes.remove("Gustavo");` // ✗ Busca el texto "Gustavo" y lo borra

Estado final de la lista: ["Paco", "Alicia"]

 Resultado final en la consola

Al llegar a la última línea:

Java

`System.out.println(nomes);`

La consola mostrará exactamente esto:

Plaintext

Xaime

[Paco, Alicia]

MATRICES

- Es una cuadrícula completa con filas y columnas

EJEMPLO EXPLICATIVO:

MATRICES BIDIMENSIONALES (`int[][]`)

Es una estructura de datos en forma de tabla o cuadrícula que organiza los elementos por filas y columnas. Se pueden hacer operaciones aritméticas.

Regla de oro para acceder:

matriz[FILA][COLUMNA] (Ambos índices empiezan siempre en 0).

Ejemplo corregido:

```
int[][] temperaturas = {  
    {25, 27, 28}, // Fila 0  
    {22, 24, 29}, // Fila 1  
    {18, 20, 23} // Fila 2  
};  
int dato = temperaturas[1][2]; // Fila 1, Columna 2 -> Vale 29
```

```
(temperaturas[1][1] + temperaturas[1][2]) / 2;
```

BinarySearch

EJEMPLO EXPLICATIVO

```
import java.util.Arrays;
```

```
public class EjemploBusqueda {  
    public static void main(String[] args) {  
        int[] numeros = {20, 5, 35, 10, 15};  
  
        // 1. ¡OBLIGATORIO! Ordenar el array primero  
        Arrays.sort(numeros);  
        // El array ahora es: {5, 10, 15, 20, 35}  
        // Índices:         0  1  2  3  4  
  
        // 2. Buscar un elemento que SÍ existe  
        int indiceEncontrado = Arrays.binarySearch(numeros, 20);  
        System.out.println("El número 20 está en el índice: " + indiceEncontrado);  
  
        // 3. Buscar un elemento que NO existe
```

```

        int indiceNoExiste = Arrays.binarySearch(numeros, 12);
        System.out.println("Resultado al buscar el 12: " + indiceNoExiste);
    }
}

```

```

int t[] = {1, 2, 5, 4, 3, 7}; //Táboa orixe
int a[] = Arrays.copyOfRange (t, 1, 4);
//a = [2, 5, 4]

```

static boolean equals

- (tipo a[], tipo b[]) compara as táboas a e b, elemento a elemento. Devolta true si son iguais e false en caso contrario.

EJEMPLO:

```

int t[] = {1, 2, 5, 4};
int s[] = {1, 2, 5, 4};
System.out.println(t==s); //Devolta false
System.out.println(Arrays.equals(t, s);
//Mostra true

```

SUBSTRING

```

public class EjemploSubstring {
    public static void main(String[] args) {
        String texto = "Sanclemente";
        // Índices:   012345678910

        // Cortamos desde la posición 3 hasta el final
        String recorte = texto.substring(3);

        System.out.println("Texto original: " + texto);
        System.out.println("Texto recortado: " + recorte);
    }
}

String texto = "Sanclemente";
// Corta desde la 0 (S) hasta la 3 (c), pero SIN incluir la 3.

```

```
String trozo = texto.substring(0, 3);
```

```
System.out.println(trozo); // Imprime: San
```

REPLACE

```
public static void main(String args[]) {  
    String cad = "Usando replace para susbtituir caracteres";  
    String novaCadea = cad.replace("r", "R");  
    System.out.println(novaCadea);  
}
```

Usando Replace paRa susbtituiR caRacteRes

COMPARETO

- Exemplos:

- "computador".compareTo("comparación")

- Devolta un enteiro > 0 porque o parámetro "comparación" está lexicográficamente en primeiro lugar.

- "casa".compareTo("coche")

- Devolta un enteiro < 0 porque a cadea "casa" que chama o método está lexicograficamente en primeiro lugar.

- "tren".compareTo("tren")

- Devolta un enteiro igual a 0, porque ambos son lexicograficamente equivalentes.

- toUpperCase(): Este método converte un string a maiúsculas.

- toLowerCase(): Este método converte un string a minúsculas.

- trim(): Este metodo quita espazos en branco no comezo e o final dun string.

EJEMPLOS DE EJERCICIOS

1. CLASE PERSONA

```
import java.util.Arrays;  
  
//SI VAMOS A UTILIZAR INTERFAZ HAY Q USAR ESTO  
  
public class Persona implements Comparable<Persona>{
```

```
// CORREGIDO: Atributos privados para cumplir los RA de DAM
(Ocultación de información)

private String nome;
private String dni;
private int idade;
private double estatura;

public Pessoa() {
    nome = "";
    dni = "0000000F";
    idade = 0;
    estatura = 0;
}

    public Pessoa(String nome, String dni, int idade, double
estatura) {
        this.nome = nome;
        this.dni = dni;
        this.idade = idade;
        this.estatura = estatura;
    }

public String getNome() {
    return this.nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public int getEdade() {
    return this.idade;
}
```

```
public void setEdade(int edade) {
    this.edade = edade;
}

public String getDni() {
    return this.dni;
}

// CORREGIDO: Nombre del parámetro cambiado a 'dni' y
asignación correcta con 'this.'
public void setDni(String dni) {
    this.dni = dni;
}

public void mostrarDatos() {
    System.out.println("Nome: " + nome + " Dni: " + dni + "
Edade: " + edade + " Estatura: " + estatura);
}

@Override
public boolean equals(Object outro) {
    Pessoa outroPessoa = (Pessoa) outro;
    if (this.estatura == outroPessoa.estatura) return true;
    return false;
}

@Override
public int compareTo(Pessoa outra) {
    /*int resultadoCom = this.dni.compareTo(outra.dni);
    System.out.println(resultadoCom);
    if (resultadoCom > 0) return 1;
    else if (resultadoCom < 0) return -1;
    return 0;*/
}
```

```

        return this.dni.compareTo(outra.dni);
    }

    public String toString() {
        return nome + ", " + dni + ", " + idade + ", " +
estatura;
    }

    public static void main(String [] args) {
        Pessoa p1 = new Pessoa();
        Pessoa p2 = new Pessoa("Manuel","1234F",45,1.67);
        // CORREGIDO: Acceso mediante getters ya que las
variables ahora son privadas
        System.out.println(p1.getNome());
        System.out.println(p2.getNome() + " " + p2.getDni() + "
" + p2.getEdade());
        Pessoa p3 = new Pessoa("Pepe","4567H",35,1.64);
        System.out.println(p3.compareTo(p3));
//          ComparadorEdadesPessoa compEdades = new
ComparadorEdadesPessoa();
//          System.out.println(compEdades.compare(p2,p3));
        Pessoa [] individuos = new Pessoa [3];
        individuos[0] = p2;
        individuos[1] = p1;
        individuos[2] = p3;
        Arrays.sort(individuos);
        System.out.println(Arrays.toString(individuos));
//          Arrays.sort(individuos, compEdades);
        System.out.println(Arrays.toString(individuos));
    }
}

```

PARA USAR COMPARABLE HAY QUE PONER EN EL CONSTRUCTOR ESTA LÍNEA

```
public class Pessoa implements Comparable<Pessoa>{
```

IMPLEMENTACION

```
@Override
public int compareTo(Pessoa outra) {
    /*int resultadoCom = this.dni.compareTo(outra.dni);
    System.out.println(resultadoCom);
    if (resultadoCom > 0) return 1;
    else if (resultadoCom < 0) return -1;
    return 0;*/
    return this.dni.compareTo(outra.dni);
}
```

NUEVO OBJETO =

objeto = new objeto

tamaño de array= objeto[] nombre_array = new objeto[tamaño]

SI QUEREMOS AÑADIR UN ELEMENTO DE LOS CREADOS CON NEW AL ARRAY=

nombre_array[posicion] = nomnre_new